

INGENIERÍA EN ENTORNOS VIRTUALES Y NEGOCIOS DIGITALES

Materia: Programación para Entornos Virtuales

Profesor(a): Ing. Anastasio Rodríguez García

Cursos NetAcad

Fundamentos de Python 2

Grupo: GIEV0351

Alumno:

Aguillon Valdez Luis Angel

Contenido

Modulo 1..... 3

Modulo 2..... 4

Modulo 3..... 9

..... 14

Modulo 4..... 14

Modulo 1

Curso

Networking Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

PE2: Módulo 1. Módulos, Paquetes, y PIP 100%

1.0 Bienvenido a Fundamentos de Python 2 3/13

1.0.1 Acerca del Curso

1.0.2 Temario

1.0.3 Preparate para el examen PCAP-31-0x

1.1 Sección 1 - Introducción a los módulos en Python 10/10

1.2 Sección 2 - Módulos Selectos de Python (math, random, platform)

1.3 Sección 3 - Módulos y Paquetes 5/10

1.4 Sección 4 - Instalador de paquetes de Python (PIP) 9/9

1.5 Finalización del Módulo 1 - Prueba del Módulo

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones 88%

PE2: Módulo 3. Programación Orientada a Objetos 100%

PE2: Módulo 4. Misceláneos 88%

Prueba Final del curso Fundamentos de Python 2 (PE2) 100%

1.0.1 Acerca del Curso



1.0 Bienvenido a Fundamentos de Python 2

Dispositivo para conectar

Este curso es el **segundo** en una serie de **dos** cursos, y te prepara para el examen de certificación [PCAP: Certified Associate in Python Programming](#) a través de Pearson VUE.

Curso

Networking Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

PE2: Módulo 1. Módulos, Paquetes, y PIP 100%

1.0 Bienvenido a Fundamentos de Python 2 3/13

1.1 Sección 1 - Introducción a los módulos en Python 10/10

1.1.1 ¿Qué es un módulo?

1.1.2 ¿Cómo hacer uso de un módulo?

1.1.3 Importando un módulo

1.1.4 Namespace

1.1.5 Importando un módulo: continuación

1.1.6 Importando un módulo: *

1.1.7 La palabra clave as

1.1.8 Alias (secomentarios)

1.1.9 RESUMEN DE LA SECCIÓN

1.1.10 CUESTIONARIO DE SECCIÓN

1.2 Sección 2 - Módulos Selectos de Python (math, random, platform) 9/9

1.3 Sección 3 - Módulos y Paquetes 5/10

1.4 Sección 4 - Instalador de paquetes de Python (PIP) 9/9

1.5 Finalización del Módulo 1 - Prueba del Módulo

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones 88%

1.1.4 Namespace

explicación será lo más breve posible.

Un **namespace** es un espacio (entendido en un contexto no físico) en el que existen algunos nombres y los nombres no entran en conflicto entre sí (es decir, no hay dos objetos diferentes con el mismo nombre). Podemos decir que cada grupo social es un namespace - el grupo tiende a nombrar a cada uno de sus miembros de una manera única (por ejemplo, los padres no darian a sus hijos el mismo nombre).

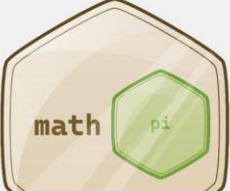
JOHNSON	SMITH	CURIE
Carl	Adam	Marie
Beverly	Brad	Irène
Brian	Frank	Ève
Sweet	Agent	Hara
Kendi	Janet	Pierre

Esta singularidad se puede lograr de muchas maneras, por ejemplo, mediante el uso de apóstrofos junto con los nombres (funcionará dentro de un grupo pequeño como una clase en una escuela) o asignando identificadores especiales a todos los miembros del grupo (el número de Seguro Social de EE. UU. es un buen ejemplo de tal práctica).

Dentro de un determinado namespace, cada nombre debe **permanecer único**. Esto puede significar que algunos nombres pueden desaparecer cuando cualquier otra entidad de un nombre ya conocido ingresa al namespace. Mostraremos como funciona y como controlarlo, pero primero, veamos a las importaciones.

Si el módulo de un nombre específico **existe y es accesible** (un módulo es de hecho un **archivo fuente de Python**), Python importa su contenido, **se hacen conocidos todos los nombres definidos en el módulo**, pero no ingresan al namespace del código.

Esto significa que puedes tener tus propias entidades llamadas



[illegible]


```
def caesar(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            shift = shift % 26
            if char.isupper():
                new_char = chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
            else:
                new_char = chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
            result += new_char
        else:
            result += char
    return result

if __name__ == '__main__':
    text = input("Ingresa una línea de texto para encriptar: ")
    shift = int(input("Ingresa un valor de cambio (1..25): "))
    while True:
        if 1 <= shift <= 25:
            break
        print("El valor de cambio debe estar entre 1 y 25")
    except ValueError:
        print("Por favor, ingresa un número entero válido.")
    cipher = caesar(text, shift)
    print("Texto encriptado:", cipher)
```

Lab 3

2.5.6 LAB Mejorando el Cifrado César

Ya estás familiarizado con el cifrado César, y es por eso que queremos que mejores el código que te mostramos recientemente.

El cifrado César original cambia cada carácter por otro a su izquierda en n , y se convierte en n , y así sucesivamente. Hagámoslo un poco más útil y permitamos que el usuario especifique el rango del rango 1..25.

Además, asegúrate de que el código conserve las mayúsculas y minúsculas (las minúsculas permanecerán en minúsculas) y todos los caracteres no alfabéticos deben permanecer intactos.

Tu tarea es escribir un programa al cual:

- Le pida al usuario una línea de texto para encriptar.
- Le pida al usuario un valor de cambio (los números enteros del rango 1..25, nota: debes obligar al usuario a ingresar un valor de cambio válido (no te envíes y no digas que los datos incorrectos te engañen)).
- Imprime el texto codificado.

Prueba tu código utilizando los datos que te proporcionamos.

Entrada de muestra:	Salida de muestra:
abcxyz123456789	abcxyz123456789
2	Output

Entrada de muestra:	Salida de muestra:
The day is cast	Sgi chr hr sars
25	Output

```
def caesar(text, shift):
    result = ""
    for char in text:
        if char.isalpha():
            shift = shift % 26
            if char.isupper():
                new_char = chr((ord(char) - ord('A') + shift) % 26 + ord('A'))
            else:
                new_char = chr((ord(char) - ord('a') + shift) % 26 + ord('a'))
            result += new_char
        else:
            result += char
    return result

if __name__ == '__main__':
    text = input("Ingresa una línea de texto para encriptar: ")
    shift = int(input("Ingresa un valor de cambio (1..25): "))
    while True:
        if 1 <= shift <= 25:
            break
        print("El valor de cambio debe estar entre 1 y 25")
    except ValueError:
        print("Por favor, ingresa un número entero válido.")
    cipher = caesar(text, shift)
    print("Texto encriptado:", cipher)
```

Lab 4

2.5.8 LAB Anagrams

Un anagrama es una nueva palabra formada al reorganizar las letras de una palabra, usando todas las letras originales exactamente una vez. Por ejemplo, las frases "red safety" y "fairy tales" son anagramas, mientras que "I am" y "You are" no lo son.

Tu tarea es escribir un programa que:

- Le pida al usuario dos textos por separado.
- Compruebe si los textos ingresados son anagramas e imprima el resultado.

Nota:

- Supongamos que dos cadenas de texto no son anagramas.
- Tratar a las letras mayúsculas y minúsculas como iguales.
- Los espacios no se tienen en cuenta durante la verificación: trátalos como inactivos.

Prueba tu código utilizando los datos que te proporcionamos.

Entrada de Muestra:	Salida de muestra:
Listen Silent	Anagramas
Output	

Entrada de Muestra:	Salida de muestra:
modern notman	No son anagramas
Output	

```
File Edit Selection View Go Run Terminal Help
ProgramacionEntornosVirtuales

EXPLORER
PROGRAMACIONENTORN...
Fundamentos1
Fundamentos2
Lab1.py
Lab2.py
Lab3.py
Lab4.py
Lab5.py
Lab6.py
Lab7.py
Lab8.py
Lab9.py
Lab10.py
Lab11.py
Pract
Fundamentos1.zip
Fundamentos2.zip

Fundamentos2 > Lab10.py >...
1 text1 = input("Ingresa el primer texto: ")
2 text2 = input("Ingresa el segundo texto: ")
3
4 # Limpiar las cadenas: sin espacios y todo a minúsculas
5 clean_text1 = text1.replace(" ", "").lower()
6 clean_text2 = text2.replace(" ", "").lower()
7
8 # Si alguno de los textos está vacío, no son anagramas
9 if len(clean_text1) == 0 or len(clean_text2) == 0:
10     print("No son anagramas")
11
12 # Al ordenar los caracteres de ambas cadenas, deben ser idénticos
13 elif sorted(clean_text1) == sorted(clean_text2):
14     print("Son anagramas")
15 else:
16     print("No son anagramas")
17
18 > Pract
19
20 PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\Microsoft\WindowsApps\Python\Python39\python.exe Fundamentos2\Lab10.py
Ingresa un valor de cambio (1..25): 1,2
No favor, Ingresa un número entero válido.
Ingresa un valor de cambio (1..25): 5
No
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\Microsoft\WindowsApps\Python\Python39\python.exe Fundamentos2\Lab10.py
Ingresa el primer texto: quitate
Ingresa el segundo texto: quitate
No son anagramas
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales>
```

Lab 5

2.8.2 LAB Leyendo enteros de manera segura

Tu tarea es escribir una función capaz de ingresar valores enteros y verificar si están dentro de un rango especificado.

La función deberá:

- Aceptar tres argumentos: una entrada, un límite inferior aceptable y un límite superior aceptable.
- Si el usuario ingresa una cadena que no es un valor entero, la función debe emitir el mensaje `Error: entrada incorrecta` y solicitar al usuario que ingrese el valor nuevamente.
- Si el usuario ingresa un número que está fuera del rango especificado, la función debe emitir el mensaje `Error: el valor no está dentro del rango permitido (el_min..el_max)` y solicitará al usuario que ingrese el valor nuevamente.
- Si el valor de entrada es válido, será regresado como resultado.

Datos de Prueba

Prueba tu código cuidadosamente.

Así es como la función debería reaccionar ante la entrada del usuario:

```
Ingresa un número entre -10 a 10: 100
Error: el valor no está dentro del rango permitido (-10..10)
Ingresa un número entre -10 a 10: asd
Error: entrada incorrecta
Ingresa un número entre -10 a 10: 1
El número es: 1
```

```
File Edit Selection View Go Run Terminal Help
ProgramacionEntornosVirtuales

EXPLORER
PROGRAMACIONENTORN...
Fundamentos1
Fundamentos2
Lab1.py
Lab2.py
Lab3.py
Lab4.py
Lab5.py
Lab6.py
Lab7.py
Lab8.py
Lab9.py
Lab10.py
Lab11.py
Pract
Fundamentos1.zip
Fundamentos2.zip

Fundamentos2 > Lab11.py >...
1 def read_int(prompt, min, max):
2     # Ciclo infinito hasta que el usuario ingrese un valor válido
3     while True:
4         try:
5             # Intentar convertir la entrada en entero
6             value = int(input(prompt))
7             # Verificar rango
8             if min <= value <= max:
9                 return value
10            else:
11                print(f"Error: el valor no está dentro del rango permitido ({min}..{max})")
12            except ValueError:
13                # Atrapar si el usuario ingresa letras u otros caracteres
14                print("Error: entrada incorrecta")
15
16 # Código de prueba
17 v = read_int("Ingresa un número entre -10 a 10: ", -10, 10)
18 print("El número es:", v)

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\Programs\Python\Python39\python.exe Fundamentos2\Lab11.py
No son anagramas
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\Programs\Python\Python39\python.exe Fundamentos2\Lab11.py
Ingresa un número entre -10 a 10: -2
El número es: -2
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\Programs\Python\Python39\python.exe Fundamentos2\Lab11.py
Ingresa un número entre -10 a 10: 6
El número es: 6
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales>
```


Modulo 3

Curso
Networking
Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

Excepciones

PE2: Módulo 3. Programación Orientada a Objetos

3.1. Sección 1 - Los fundamentos de la POO

3.1.1 Los conceptos básicos del enfoque orientado a objetos

3.1.2 Enfoque procedimental versus el enfoque orientado a objetos

3.1.3 Jerarquía de clases

3.1.4 ¿Qué es un objeto?

3.1.5 Herencia

3.1.6 ¿Qué contiene un objeto?

3.1.7 Tu primera clase

3.1.8 Tu primer objeto

3.1.9 RESUMEN DE SECCIÓN

3.1.10 CUESTIONARIO DE SECCIÓN

3.2. Sección 2 - Un corto viaje desde el enfoque procedimental hacia el orientado a objetos

3.3. Sección 3 - POO: Propiedades

3.4. Sección 4 - POO: Métodos

3.5. Sección 5 - Fundamentos de la POO: Herencia

3.6. Sección 6 - Más sobre excepciones

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.1.3 Jerarquía de clases

La palabra **clases** tiene muchos significados, pero no todos son compatibles con las ideas que queremos discutir aquí. La clase que nos concierne es como una categoría, como resultado de similitudes definidas con precisión. Intentaremos señalar algunas clases que son buenos ejemplos de este concepto.

```
graph BT; VT[Vehículos Terrestres]; VA[Vehículos Acuáticos]; VAero[Vehículos Aéreos]; VE[Vehículos Espaciales]; VCR[Vehículos con ruedas]; VCP[Vehículos con pistas]; A[Aerodispositivos]; V[Vehículos]; VCR --> VT; VCR --> VA; VCP --> VT; VCP --> VA; A --> VAero; A --> VE; VT --> V; VA --> V; VAero --> V; VE --> V;
```

Curso
Networking
Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

Excepciones

PE2: Módulo 3. Programación Orientada a Objetos

3.1. Sección 1 - Los fundamentos de la POO

3.1.1 Los conceptos básicos del enfoque orientado a objetos

3.1.2 Enfoque procedimental versus el enfoque orientado a objetos

3.1.3 Jerarquía de clases

3.1.4 ¿Qué es un objeto?

3.1.5 Herencia

3.1.6 ¿Qué contiene un objeto?

3.1.7 Tu primera clase

3.1.8 Tu primer objeto

3.1.9 RESUMEN DE SECCIÓN

3.1.10 CUESTIONARIO DE SECCIÓN

3.2. Sección 2 - Un corto viaje desde el enfoque procedimental hacia el orientado a objetos

3.3. Sección 3 - POO: Propiedades

3.4. Sección 4 - POO: Métodos

3.5. Sección 5 - Fundamentos de la POO: Herencia

3.6. Sección 6 - Más sobre excepciones

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.1.7 Tu primera clase

La programación orientada a objetos es el **arte de definir y expandir clases**. Una clase es un modelo de una parte muy específica de la realidad, que refleja las propiedades y actividades que se encuentran en el mundo real.

Las clases definidas al principio son demasiado generales e imprecisas para cubrir el mayor número posible de casos reales.

No hay obstáculo para definir nuevas subclases más precisas. Heredarán todo de su superclase, por lo que el trabajo que se utilizó para su creación no se desperdicia.

La nueva clase puede agregar nuevas propiedades y nuevas actividades y, por lo tanto, puede ser más útil en aplicaciones específicas. Obviamente, se puede usar como una superclase para cualquier número de subclases recién creadas.

El proceso no necesita tener un final. Puedes crear tantas clases como necesites.

La clase que se define no tiene nada que ver con el objeto: **la existencia de una clase no significa que ninguno de los objetos compatibles se creará automáticamente**. La clase en sí misma no puede crear un objeto: debes crearlo tu mismo y Python te permite hacerlo.

Es hora de definir la clase más simple y crear un objeto. Analiza el siguiente ejemplo:

```
1 class TheSimplestClass:
2     pass
3
```

Hemos definido una clase. La clase es bastante pobre: no contiene propiedades ni actividades. Esta **vacía**, pero eso no importa por ahora. Cuanto más simple sea la clase, mejor para nuestros propósitos.

La **definición comienza con la palabra clave reservada `class`**. La palabra clave reservada es seguida por un **identificador que le dará nombre a la clase** (nota: no lo confundas con el nombre del objeto: estas son dos cosas diferentes).

A continuación, se agregan **dos puntos**, como clases, como funciones, forman su propio bloque anidado. El contenido dentro del bloque define todas las propiedades y actividades de la clase.

La palabra clave reservada `pass` llena la clase con nada. No contiene ningún método ni propiedades.

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 2. Cadenas, Métodos de Cadenas y listas, Excepciones88%

PE2: Módulo 3. Programación Orientada a Objetos100%

3.1. Sección 1 - Los fundamentos de la POO10 / 10

3.2. Sección 2 - Un corto viaje desde el enfoque procedural hacia el orientado a objetos10 / 10

3.2.1 ¿Qué es una pila?

3.2.2 La pila: el enfoque procedural

3.2.3 La pila: el enfoque procedural versus el enfoque orientado a objetos

3.2.4 La pila: el enfoque orientado a objetos

3.2.5 El enfoque orientado a objetos: una pila desde cero

3.2.6 RESUMEN DE SECCIÓN

3.2.7 CUESTIONARIO DE SECCIÓN

3.2.8 LAB Pila contadora

3.2.9 LAB Cotas alias FIFO

3.2.10 LAB Cotas alias FIFO: parte 2

3.3. Sección 3 - POO: Propiedades5 / 5

3.4. Sección 4 - POO: Métodos10 / 10

3.5. Sección 5 - Fundamentos de la POO: Herencia11 / 11

3.6. Sección 6 - Más sobre excepciones6 / 6

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.2.5 El enfoque orientado a objetos: una pila desde cero

Ahora es el momento de que las dos funciones (métodos) implementen las operaciones push y pop. Python supone que una función de este tipo debería estar **inmersa dentro del cuerpo de la clase**, como el constructor.

Queremos invocar estas funciones para `self.stack` y `self.val` valores de la pila. Esto significa que ambos deben ser accesibles para el usuario de la clase (en contraste con la lista previamente construida, que está oculta para los usuarios de la clase ordinaria).

Tal componente es llamado **público**, por ello **no puede comenzar su nombre con dos (o más) guilanes bajos**. Hay un requisito más: **el nombre no debe tener más de un guión bajo**. Dado que ningún guión bajo final cumple completamente con el requisito, puedes suponer que el nombre es aceptable.

Las funciones en sí son simples. Echa un vistazo

```
1 class Stack:
2     def __init__(self):
3         self.__stack_list = []
4
5     def push(self, val):
6         self.__stack_list.append(val)
7
8     def pop(self):
9         val = self.__stack_list[-1]
10        del self.__stack_list[-1]
11        return val
```

Sin embargo, hay algo realmente extraño en el código. Las funciones parecen familiares, pero tienen más parámetros que sus contrapartes procedimentales.

Aquí, ambas funciones tienen un parámetro llamado `self` en la primera posición de la lista de parámetros.

Cisco Networking AcademyPythonFundamentos de Python 2

Course OutlineResources

Search course outline

PE2: Módulo 3. Programación Orientada a Objetos100%

3.1. Sección 1 - Los fundamentos de la POO10 / 10

3.2. Sección 2 - Un corto viaje desde el enfoque procedural hacia el orientado a objetos10 / 10

3.3. Sección 3 - POO: Propiedades5 / 5

3.4. Sección 4 - POO: Métodos10 / 10

3.4.1 Métodos a detalle

3.4.2 La vida al interior de las clases y objetos

3.4.3 Reflexión e introspección

3.4.4 Investigando Clases

3.4.5 RESUMEN DE SECCIÓN

3.4.6 CUESTIONARIO DE SECCIÓN

3.4.7 LAB La clase Timer

3.4.8 LAB Días de la semana

3.4.9 LAB Puntos en un plano

3.4.10 LAB Triángulo

3.5. Sección 5 - Fundamentos de la POO: Herencia11 / 11

3.6. Sección 6 - Más sobre excepciones6 / 6

3.7. Finalización del Módulo 3 - Prueba del Módulo

3.4.2 La vida al interior de las clases y objetos

Cada clase de Python y cada objeto de Python está pre-equipado con un conjunto de atributos útiles que pueden usarse para examinar sus capacidades.

Ya conoces uno de estos: es la propiedad `__dict__`.

Observemos como esta propiedad trata con los métodos: mira el código en el editor.

```
1 class Classy:
2     var1 = 1
3     def __init__(self):
4         self.var = 2
5
6     def method(self):
7         pass
8
9     def hidden(self):
10        pass
11
12 obj = Classy()
```

Ejecútalo para ver que produce. Verifica el resultado.

Encuentra todos los métodos y atributos definidos. Localiza el contexto en el que existen: dentro del objeto o dentro de la clase.

`__dict__` es un diccionario. Otra propiedad incorporada que vale la pena mencionar es una cadena llamada `__name__`.

La propiedad contiene el **nombre de la clase**. No es nada emocionante, es solo una cadena.


```
1 class Queue:
2     def __init__(self, elem):
3         self._queue.insert(0, elem)
4
5     def get(self):
6         # elimina y devuelve el último elemento (el primero en llegar)
7         if not self._queue:
8             raise QueueError("Queue error")
9         return self._queue.pop()
10
11 # Código de prueba (salida esperada: 1, perro, False, Queue error)
12 que = Queue()
13 que.put(1)
14 que.put("perro")
15 que.put("False")
16 try:
17     for i in range(4):
18         print(que.get())
19 except QueueError:
20     print("Queue error")
```

perro
False
Queue error

Lab 8

3.2.10 LAB Colas alias FIFO: parte 2

Tu tarea es extender ligeramente las capacidades de la clase `Queue`. Queremos que tenga un método sin parámetros que devuelva `True` si la cola está vacía y `False` de lo contrario.

Completa el código que te proporcionamos en el editor. Ejecútalo para comprobar si genera un resultado similar al nuestro.

A continuación, puedes copiar el código que usamos en el laboratorio anterior

Revisar

Salida esperada

```
1  
perro  
False  
Cola vacia
```

```
1 class QueueError(Exception):
2     pass
3
4 class Queue:
5     def __init__(self):
6         self._queue = []
7
8     def put(self, elem):
9         self._queue.insert(0, elem)
10
11     def get(self):
12         if not self._queue:
13             raise QueueError("Queue error")
14         return self._queue.pop()
15
16     def is_empty(self):
17         return len(self._queue) == 0
18
19 # Código de prueba (salida esperada: 1, perro, False, Cola vacia)
20 que = Queue()
21 que.put(1)
22 print(que.get())
23 print(que.is_empty())
```

1
perro
False
Cola vacia

Lab 9

3.4.7

LAB

La clase Timer

New Component Title

Necesitamos una clase capaz de contar segundos. ¿Fácil? No es tan fácil como podrías pensar, ya que tendremos algunos requisitos específicos.

Léelos con atención, ya que la clase sobre la que escribes se utilizará para lanzar cohetes en misiones internacionales a Marte. Es una gran responsabilidad. ¿Contamos contigo?

Tu clase se llamará `Timer` (temporizador en español). Su constructor acepta tres argumentos que representan **horas** (un valor del rango [0, 23]), **minutos** (del rango [0, 59]) y **segundos** (del rango [0, 59]).

Cero es el valor predeterminado para todos los parámetros anteriores. No es necesario realizar ninguna comprobación de validación.

La clase en sí debería proporcionar las siguientes facilidades:

- Los objetos de la clase deben ser "imprimibles", es decir, deben poder convertirse implícitamente en cadenas de la siguiente forma: "hh:mm:ss", con ceros a la izquierda agregados cuando cualquiera de los valores es menor que 10.
- La clase debe estar equipada con métodos sin parámetros llamados `next_second()` y `prev_second()`, incrementando el tiempo almacenado dentro de los objetos en +1/-1 segundos respectivamente.

Emplea las siguientes sugerencias:

- Todas las propiedades del objeto deben ser privadas.
- Considera escribir una función separada (no un método) para formatear la cadena con el tiempo.

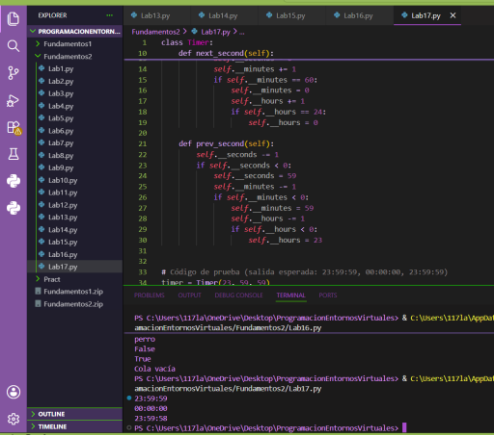
Completa la plantilla que te proporcionamos en el editor. Ejecuta tu código y comprueba si el resultado es el mismo que el nuestro.

Salida Esperada

23:59:59

00:00:00

23:59:59



Lab 10

3.4.8 LAB Días de la semana

Tu tarea es implementar una clase llamada `Weeker`. Si, tus ojos no te engañan, este nombre proviene del hecho de que los objetos de esta clase podrán almacenar y manipular los días de la semana.

El constructor de la clase acepta un argumento: una cadena. La cadena representa el nombre del día de la semana y los únicos valores aceptables deben provenir del siguiente conjunto:

```
Mon Tue Wed Thu Fri Sat Sun
```

Invocar al constructor con un argumento desde fuera de este conjunto debería generar la excepción `WeekDayError` (defínela tu mismo; no te preocupes, pronto hablaremos sobre la naturaleza objetiva de las excepciones). La clase debe proporcionar las siguientes facilidades:

- Los objetos de la clase deben ser "imprimibles", es decir, deben poder convertirse implícitamente en cadenas de la misma forma que los argumentos del constructor.
- La clase debe estar equipada con métodos de un parámetro llamados `add_days(n)` y `subtract_days(n)`, siendo `n` un número entero que actualiza el día de la semana almacenado dentro del objeto mediante el número de días indicado, hacia adelante o hacia atrás.
- Todas las propiedades del objeto deben ser privadas.

Completa la plantilla que te proporcionamos en el editor, ejecuta su código y verifica si tu salida se ve igual que la nuestra. Completa la plantilla que te proporcionamos en el editor, ejecuta su código y verifica si tu salida se ve igual que la nuestra.

Salida Esperada

Lun

Mar

Dom

Lo siento, no puedo atender tu solicitud.

```
1 class weekender:
2     __days = ["Sun", "Mon", "Tue", "Wed", "Thu", "Fri", "Sat"]
3
4     def __init__(self, day):
5         if day not in self.__days:
6             raise ValueError("No es un día")
7         self.__day = self.__days.index(day) # almacenamos índice (0-6)
8
9     def __str__(self):
10        return self.__days[self.__day]
11
12    def add_days(self, n):
13        self.__day = (self.__day + n) % 7
14
15    def subtract_days(self, n):
16        self.__day = (self.__day - n) % 7
17
18    # Código de prueba (salida esperada: Sun, Mon, Tue, 'Lo siento, no puedo atender tu solicitud.')
19    if __name__ == '__main__':
20        weekend = weekender("Sun")
21        print(weekend)
22        weekend.add_days(1)
23        print(weekend)
24        weekend.subtract_days(1)
25        print(weekend)
```

Modulo 4

4.1.1 Generadores: donde encontrarlos

Generador - ¿Con qué asocias esta palabra? Quizás se refiere a algún dispositivo electrónico. O tal vez se refiere a una máquina pesada diseñada para producir energía eléctrica u otra cosa.

Un generador de Python es un fragmento de código especializado capaz de producir una serie de valores y controlar el proceso de iteración. Esta es la razón por la cual los generadores a menudo se llaman **Iteradores**, y aunque hay quienes pueden encontrar una diferencia entre estos dos, aquí los trataremos como uno mismo.

```
1 for i in range(5):
2     print(i)
```

Puede que no te hayas dado cuenta, pero te has topado con generadores muchas, muchas veces antes. Echa un vistazo al fragmento de código:

La función `range()` es un generador, la cual también es un iterador.

4.1.4 Más acerca de listas por comprensión

Debes poder recordar las reglas que rigen la creación y el uso de un fenómeno de Python llamado **listas por comprensión**: una forma simple de crear listas y sus contenidos.

En caso de que lo necesitas, te proporcionamos un recordatorio en el editor.

```
1 list_1 = []
2 for ex in range(5):
3     list_1.append(10 ** ex)
4
5 list_2 = [10 ** ex for ex in range(5)]
6
7 print(list_1)
8 print(list_2)
```

Existen dos partes dentro del código, ambas crean una lista que contiene algunas de las primeras potencias naturales de diez.

La primera parte utiliza una forma rutinaria del bucle `for`, mientras que la segunda hace uso de listas por comprensión y construye la lista en el momento, sin necesidad de un bucle o cualquier otro código.

Pareciera que la lista se crea dentro de sí misma; esto es falso, ya que Python tiene que realizar casi las mismas operaciones que en la primera parte, pero el segundo formalismo es simplemente más elegante y le evita al lector cualquier detalle innecesario.

El ejemplo genera dos líneas idénticas que contienen el siguiente texto:

```
Output
```

Curso

Networking Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

PE2: Módulo 3. Programación Orientada a Objetos

PE2: Módulo 4. Misceláneos

4.1. Sección 1 - generadores, iteradores y cierres

4.2. Sección 2 - Archivos (flujos de archivos, procesamiento de archivos, diagnóstico de problemas de flujos)

4.3. Sección 3 - Trabajando con archivos reales

4.4. Sección 4 - El módulo os - interactuando con el sistema operativo

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario

4.2.3 Flujos de archivos (streams)

4.2.4 Manejo de archivos

Python supone que **cada archivo está oculto detrás de un objeto de una clase adecuada**. Por supuesto, es difícil no preguntar cómo interpretar la palabra adecuada.

Los archivos se pueden procesar de muchas maneras diferentes: algunos dependen del contenido del archivo, otros de las intenciones del programador.

En cualquier caso, diferentes archivos pueden requerir diferentes conjuntos de operaciones y comportarse de diferentes maneras.

Un objeto de una clase adecuada es **creado cuando abres el archivo y lo aniquilas al momento de cerrarlo**.

Entre estos dos eventos, puedes usar el objeto para especificar que operaciones se deben realizar en un stream en particular. Las operaciones que puedes usar están impuestas por la **forma en que abriste el archivo**.

En general, el objeto proviene de una de las clases que se muestran aquí:

IOBase

RawIOBase

BufferedIOBase

TextIOBase

Nota: nunca se utiliza el constructor para dar vida a estos objetos. La única forma de obtenerlos es invocar la función llamada `open()`

Curso

Networking Academy

Python

Fundamentos de Python 2

Course Outline

Resources

Search course outline

PE2: Módulo 4. Misceláneos

4.1. Sección 1 - generadores, iteradores y cierres

4.2. Sección 2 - Archivos (flujos de archivos, procesamiento de archivos, diagnóstico de problemas de flujos)

4.3. Sección 3 - Trabajando con archivos reales

4.4. Sección 4 - El módulo os - interactuando con el sistema operativo

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario

4.3.5 ¿Qué es un bytearray?

4.3.5 ¿Qué es un bytearray?

Antes de comenzar a hablar sobre archivos binarios, tenemos que informarte sobre una de las **clases especializadas que Python usa para almacenar datos amorfos**.

Los datos amorfos son datos que no tienen forma específica, son solo una serie de bytes.

Esto no significa que estos bytes no puedan tener su propio significado o que no puedan representar ningún objeto útil, por ejemplo, gráficos de mapa de bits.

Lo más importante de esto es que en el lugar donde tenemos contacto con los datos, no podemos, o simplemente no queremos, saber nada al respecto.

Los datos amorfos no pueden almacenarse utilizando ninguno de los medios presentados anteriormente: no son cadenas ni listas.

Debe haber un contenedor especial capaz de manejar dichos datos.

Python tiene más de un contenedor, uno de ellos es una **clase especializada llamada bytearray**, como su nombre indica, es un **arreglo que contiene bytes (amorfos)**.

Si deseas tener dicho contenedor, por ejemplo, para leer una imagen de mapa de bits y procesarla de alguna manera, debes crearlo explícitamente, utilizando uno de los constructores disponibles.

Observa:

```
1 data = bytearray(10)
2
```

Tal invocación crea un objeto bytearray capaz de almacenar diez bytes.

Nota: dicho constructor **llena todo el arreglo con ceros**.

Bytearrays se asemejan a listas en muchos aspectos. Por ejemplo, son **mutables**, son susceptibles a la función `len()` y puedes acceder a cualquiera de sus elementos usando indexación convencional.

Existe una limitación importante: **no debes establecer ningún elemento del arreglo de bytes con un valor que no sea un entero** (violar

Oswin
Networking
Academy

Course Outline

Fundamentos de Python 2

Resources

4.7. Finalización del Módulo 4 - Prueba del Módulo

Prueba Final del curso Fundamentos de Python 2 (PE2)

4.3. Sección 3 - Trabajando con archivos reales

4.4. Sección 4 - El módulo os - Interactuando con el sistema operativo

4.5. Sección 5 - el módulo datetime - trabajar con funciones relacionadas con la hora y la fecha

4.6. Sección 6 - El módulo calendar - trabajando con funciones relacionadas con el calendario

4.6.1 Introducción al módulo calendar

4.6.2 Tu primer calendario

4.6.3 El calendario para un mes específico

4.6.4 La función strftime weekday()

4.6.5 La función weekday()

4.6.6 La función veeekend()

4.6.7 ¿Cómo comprobamos si un año es bisiesto?

4.6.8 Clases para la creación de calendarios

4.6.9 Creando un objeto Calendar

4.6.10 El método __str__(self)

4.6.11 Otros métodos que retornan o devuelven iteradores

4.6.12 El método monthdays(calendar())

4.6.13 LAB El módulo calendar

4.6.14 RESUMEN DE SECCIÓN

4.6.15 CUESTIONARIO DE SECCIÓN

4.6.2 Tu primer calendario

Comenzarás tu aventura con el módulo `calendar` con una función simple llamada `calendar`, que te permite mostrar el calendario para todo el año. Veamos cómo usarlo para mostrar el calendario de 2020. Ejecuta el código en el editor y ve qué sucede.

```
import calendar\nprint(calendar.calendar(2020))
```

El resultado mostrado es similar al resultado del comando `cal` disponible en Unix. Si deseas cambiar el formato de calendario predeterminado, puedes utilizar los siguientes parámetros:

- ancho de la columna de fecha (por defecto 2).
- número de líneas por semana (por defecto 1).
- número de espacios entre las columnas del mes (por defecto 6).
- número de columnas (por defecto 3).

La función de calendario requiere que se especifique el año, mientras que los otros parámetros responsables del formato son opcionales. Te recomendamos que pruebes estos parámetros tú mismo.

Una buena alternativa a la función anterior es la función llamada `prcal`, que también toma los mismos parámetros que la función `calendar`.

4.3.8 LAB Histograma de frecuencia de caracteres

Tu tarea es escribir un programa que:

- Pida al usuario el nombre del archivo de entrada.
- Lea el archivo (si es posible) y cuente todas las letras latinas (las letras mayúsculas y minúsculas se tratan como iguales).
- Imprima un histograma simple en orden alfabético (solo se deben presentar recuentos distintos de cero).

Crea un archivo de prueba para tu código y verifica si tu histograma contiene resultados válidos.

Suponiendo que el archivo de prueba contiene solo una línea con:

aBc

samplefile.txt

New Component Title

El resultado esperado debería verse de la siguiente manera:

```
a -> 1
b -> 1
c -> 1
```

Output


```
1 # Solicita el nombre del archivo, cuenta letras latinas (mayúsculas/minúsculas igual) e imprime histograma alfabético
2
3 nombre_archivo = input("Introduce el nombre del archivo: ")
4
5 try:
6     with open(nombre_archivo, "r", encoding="utf-8") as f:
7         texto = f.read()
8     except FileNotFoundError:
9         print("Archivo no encontrado.")
10        exit()
11
12 # contar solo letras del alfabeto latino (a-z)
13 conteo = {}
14 for letra in texto.lower():
15     if 'a' <= letra <= 'z':
16         conteo[letra] = conteo.get(letra, 0) + 1
17
18 # imprimir en orden alfabético solo las que aparecen
19 for letra in sorted(conteo.keys()):
20     print(f"{letra} -> {conteo[letra]}")
```

Lab 12

4.3.9 LAB Histograma de frecuencia de caracteres ordenado

El código anterior necesita ser mejorado. Está bien, pero tiene que ser mejor. Tu tarea es hacer algunas modificaciones, que generen los siguientes resultados:

- El histograma de salida se ordenará en función de la frecuencia de los caracteres (el contador más grande debe presentarse primero).
- El histograma debe enviarse a un archivo con el mismo nombre que el de entrada, pero con la extensión ".hist" (debe concatenarse con el nombre original).

Suponiendo que el archivo de prueba contiene solo una línea con:

```
cabada
```

El resultado esperado debería verse de la siguiente manera:

```
a -> 3
b -> 2
c -> 1
```

Consejo: Emplea una `lambda` para cambiar el ordenamiento.

```
13
14 conteo = {}
15 for letra in texto.lower():
16     if 'a' <= letra <= 'z':
17         conteo[letra] = conteo.get(letra, 0) + 1
18
19 # ordenar por frecuencia descendente (y alfabéticamente para iguales)
20 items_ordenados = sorted(conteo.items(), key=lambda x: (-x[1], x[0]))
21
22 # nombre de salida: mismo nombre base + ".hist"
23 letra = os.path.splitext(nombre_entrada)
24 nombre_salida = base + ".hist"
25
26 with open(nombre_salida, "w", encoding="utf-8") as salida:
27     for letra, cant in items_ordenados:
28         salida.write(f"{letra} -> {cant}\n")
29
30 print(f"Histograma guardado en {nombre_salida}")
```

Lab 13

4.3.10 LAB Evaluando los resultados de los estudiantes

El profesor Jekyll dirige clases con estudiantes y regularmente toma notas en un archivo de texto. Cada línea del archivo contiene 3 elementos: el nombre del alumno, el apellido del alumno y el número de puntos que el alumno recibió durante ciertas clases.

Los elementos están separados con espacios en blanco. Cada estudiante puede aparecer más de una vez dentro del archivo del profesor Jekyll.

El archivo puede tener el siguiente aspecto:

```
John  Saketh  9
Anna  Boleyn  4.5
John  Saketh  2
Anna  Boleyn  11
Andrew Cox  1.5
```

sample.txt

New Component Title

Tu tarea es escribir un programa que:

- Pida al usuario el nombre del archivo del profesor Jekyll.
- Lee el contenido del archivo y cuenta la suma de los puntos recibidos por cada estudiante.
- Imprima un informe simple (pero ordenado), como este:

```
Andrew Cox 1.5
Anna Boleyn 11.5
John Saketh 7.5
```

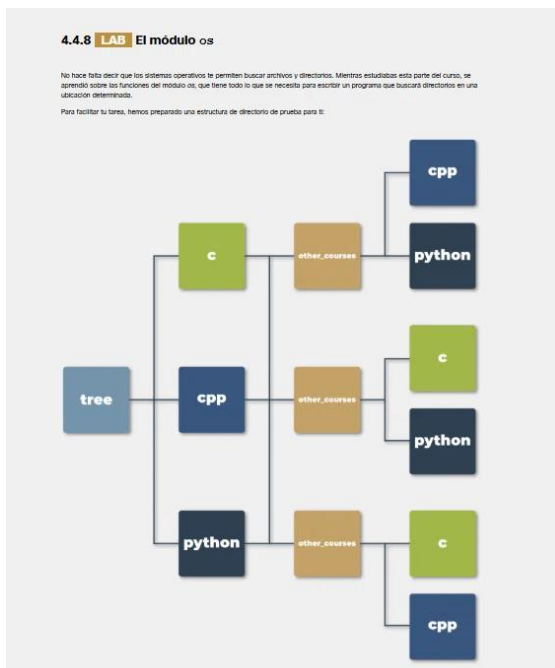
```

EXPLORER
PROGRAMACIONINFORMOS
Fundamentos1
Fundamentos2
Lab1.py
Lab2.py
Lab3.py
Lab4.py
Lab5.py
Lab6.py
Lab7.py
Lab8.py
Lab9.py
Lab10.py
Lab11.py
Lab12.py
Lab13.py
Lab14.py
Lab15.py
Lab16.py
Lab17.py
Lab18.py
Lab19.py
Lab20.py
Lab21.py
Fundamentos1.zip
Fundamentos2.zip
OUTLINE
TIMELINE

lab21.py
Fundamentos2 > lab21.py >
1 # Lee archivo con nombre, apellido y nota; suma por estudiante y genera informe ordenado.
2
3 nombre_archivo = input("Introduce el nombre del archivo: ")
4
5 try:
6     with open(nombre_archivo, 'r', encoding='utf-8') as f:
7         lineas = f.readlines()
8     except FileNotFoundError:
9         print("Archivo no encontrado.")
10        exit()
11
12 # Diccionario con clave (nombre, apellido) -> suma de puntos
13 suma = {}
14 for linea in lineas:
15     partes = linea.strip().split()
16     if len(partes) < 3:
17         continue # línea sin fórmula
18     nombre, apellido, nota_str = partes[0], partes[1], partes[2]
19     try:
20         nota = float(nota_str)
21     except ValueError:
22         continue
23     clave = (nombre, apellido)
24
25 PS C:\Users\1171a\OneDrive\Desktop\ProgramacionInformos\virtuales> & C:\Users\1171a\AppData\Local\Programs\Python\Python39\python.exe C:\Users\1171a\OneDrive\Desktop\ProgramacionInformos\virtuales\Fundamentos2\lab21.py
Introduce el nombre del archivo: Fundamentos2\lab21.py
Introduce el nombre del archivo:
nombre_archivo = input("Introduce el nombre del archivo: ")
KeyboardInterrupt
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionInformos\virtuales> C:\Users\1171a\AppData\Local\Programs\Python\Python39\python.exe C:\Users\1171a\OneDrive\Desktop\ProgramacionInformos\virtuales\Fundamentos2\lab21.py
Introduce el nombre del archivo:
Introduce el nombre del archivo:
Archivo no encontrado.
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionInformos\virtuales>

```

Lab 14



Tu programa debe cumplir con los siguientes requisitos:

1. Escribe una función o método llamado `find` que tome dos argumentos llamados `path` y `dir`. El argumento `path` debe aceptar una ruta relativa o absoluta a un directorio donde debe comenzar la búsqueda, mientras que el argumento `dir` debe ser el nombre de un directorio en el que deseas encontrar la ruta dada. Tu programa debería mostrar las rutas absolutas si encuentra un directorio con el nombre dado.
2. La búsqueda en el directorio debe realizarse de forma recursiva. Esto significa que la búsqueda también debe incluir todos los subdirectorios en la ruta dada.

Entrada de muestra:

```
path = "../tree", dir = "python"
```

Salida de muestra:

```

.../tree/python
.../tree/cpp/other_courses/python
.../tree/c/other_courses/python

```

```
1 import os
2
3 def find(path, dir_name):
4     """
5     Busca recursivamente todos los directorios con nombre 'dir_name' dentro de 'path'.
6     Imprime las rutas absolutas de cada uno.
7     """
8     # Convertir a ruta absoluta para salida limpia
9     path_abs = os.path.abspath(path)
10
11     for root, dirs, files in os.walk(path_abs):
12         # Verificar si el directorio actual (root) tiene el nombre buscado
13         if os.path.basename(root) == dir_name:
14             print(root)
15             # También se puede buscar en los subdirectorios, pero os.walk ya los recorre
16
17 # Ejemplo de uso (descomentar para probar)
18 # find("../tree", "python")
```

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\ProgramacionEntornosVirtuales\Fundamentos2\Lab21.py

File "C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales\Fundamentos2\Lab21.py", line 3, in nombre_archivo = input("Introduce el nombre del archivo: ")

KeyboardInterrupt

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> C:\Users\1171a\AppData\Local\ProgramacionEntornosVirtuales\Fundamentos2\Lab21.py

Introduce el nombre del archivo: hola

Archivo no encontrado.

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\ProgramacionEntornosVirtuales\Fundamentos2\Lab22.py

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales>

Lab 15

4.5.20 LAB Los módulos datetime y time

Durante este curso, has aprendido sobre el método `strftime`, que requiere conocimiento de las directivas para crear un formato. Ahora es el momento de poner en práctica estas directivas.

Por cierto, tendrás la oportunidad de practicar el trabajo con documentación, porque tendrás que encontrar directivas que aún no conoces.

Aquí está tu tarea:

Escibe un programa que cree un objeto `datetime` para el 4 de noviembre de 2020, 14:53:00. El objeto creado debe llamar al método `strftime` con el formato apropiado para mostrar el siguiente resultado.

```
2020/11/04 14:53:00
20/November/04 14:53:00 PM
Wed, 2020 Nov 04
Wednesday, 2020 November 04
Día de la semana: 3
Día del año: 309
Número de semana en el año: 44
```

Output

Nota: Cada línea de resultado debe crearse llamando al método `strftime` con al menos una directiva en el argumento de formato.

```
1 from datetime import datetime
2
3 # Crear el objeto datetime para el 4 de noviembre de 2020, 14:53:00
4 dt = datetime(2020, 11, 4, 14, 53, 0)
5
6 # Mostrar en los formatos solicitados
7 print(dt.strftime("%w/%m/%d %H:%M:%S"))
8 print(dt.strftime("%p/%m/%d %H:%M:%S %p"))
9 print(dt.strftime("%a, %Y %b %d"))
10 print(dt.strftime("%A, %Y %b %d"))
11 print(f"Día de la semana: {dt.isoweekday()}") # 3 (miércoles según ISO)
12 print(f"Día del año: {dt.timetuple().tm_yday}") # 309
13 print(f"Número de semana en el año: {dt.strftime('%W')}") # 44 (semana con domingo como primer día)
```

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\ProgramacionEntornosVirtuales\Fundamentos2\Lab22.py

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\ProgramacionEntornosVirtuales\Fundamentos2\Lab23.py

2020/11/04 14:53:00

20/November/04 14:53:00 PM

Wed, 2020 Nov 04

Wednesday, 2020 November 04

Día de la semana: 3

Día del año: 309

Número de semana en el año: 44

PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales>

Lab 16

4.6.13 LAB El módulo calendar

Durante este curso, echamos un breve vistazo a la clase `Calendar`. Tu tarea ahora es ampliar su funcionalidad con un nuevo método llamado `count_weekday_in_year`, que toma un año y un día de la semana como parámetros, y luego devuelve el número de ocurrencias de un día de la semana específico en el año.

Utiliza los siguientes consejos:

- Crea una clase llamada `MyCalendar` que se extiende de la clase `Calendar`.
- Crea el método `count_weekday_in_year` con los parámetros de `year` y `weekday`. El parámetro `weekday` debe tener un valor entre 0 y 6, donde 0 es el lunes y 6 es el domingo. El método debe devolver el número de días como un número entero.
- En la implementación, usa el método `monthdays2calendar` de la clase `Calendar`.

Los siguientes son resultados esperados de ejemplo:

Argumentos de muestra

```
year=2019, weekday=0
```

Output

New Component Title

Salida esperada

```
52
```

Output

Argumentos de muestra

```
year=2000, weekday=6
```

Salida esperada

```
53
```

Output

```
EXPLORER
PROGRAMACIONENTORNOS...
Fundamentos1
Fundamentos2
  Lab1.py
  Lab2.py
  Lab3.py
  Lab4.py
  Lab5.py
  Lab6.py
  Lab7.py
  Lab8.py
  Lab9.py
  Lab10.py
  Lab11.py
  Lab12.py
  Lab13.py
  Lab14.py
  Lab15.py
  Lab16.py
  Lab17.py
  Lab18.py
  Lab19.py
  Lab20.py
  Lab21.py
  Lab22.py
  Lab23.py
  Lab24.py
  Pract
Fundamentos1.zip
Fundamentos2.zip
OUTLINE
TIMELINE

Lab24.py X
Fundamentos2 > Lab24.py > ...
1 import calendar
2
3 class MyCalendar(calendar.Calendar):
4     def count_weekday_in_year(self, year, weekday):
5         """
6         Devuelve el número de ocurrencias del día de la semana (0=lunes, ..., 6=domingo)
7         en el año dado.
8         """
9         count = 0
10        for month in range(1, 13):
11            # monthdays2calendar devuelve una lista de semanas, cada semana es una lista
12            # de tuplas (día, día semana), donde día=0 significa fuera del mes.
13            weeks = self.monthdays2calendar(year, month)
14            for week in weeks:
15                for day, wd in week:
16                    if day != 0 and wd == weekday:
17                        count += 1
18        return count
19
20
21 # Ejemplo de prueba (descomentar para ejecutar)
22 if __name__ == "__main__":
23     mc = MyCalendar()
24
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\Programs\Python\
amacionEntornosVirtuales\Fundamentos2\Lab23.py
Wed, 2020 Nov 04
Wednesday, 2020 November 04
Día de la semana: 3
Día del año: 309
Número de semana en el año: 44
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales> & C:\Users\1171a\AppData\Local\Programs\Python\
amacionEntornosVirtuales\Fundamentos2\Lab24.py
52
53
PS C:\Users\1171a\OneDrive\Desktop\ProgramacionEntornosVirtuales>
```